10_(s+2)(s+5)

October 30, 2026

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import control as ct
import scipy.signal as signal
```

[3]:
```
K = 10
G1 = ct.tf([1], [1, 2])
G2 = ct.tf([1], [1, 5])
G3 = 1

# Gain of the system K
print(f'The gain of the system K: {K}')

# Actual Plant
G = K * G1 * G2 * G3
print(f'The transfer function of the system G(s): {G}')

# Feedback Loop Transfer Function
H = 1
print(f'The transfer function of the feedback loop H(s): {H}')

# Open Loop Transfer Funtion
L = G * H
print(f'The transfer function of the open loop system L(s): {L}')

# Closed Loop Transfer Funtion
CL = ct.feedback(G, H)
print(f'The transfer function of the closed loop system CL(s): {CL}')
```

```
The gain of the system K: 10
The transfer function of the system G(s): <TransferFunction>: sys[17]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

        10
  --------------
  s^2 + 7 s + 10
The transfer function of the feedback loop H(s): 1
```

```
The transfer function of the open loop system L(s): <TransferFunction>: sys[19]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

        10
  --------------
  s^2 + 7 s + 10
The transfer function of the closed loop system CL(s): <TransferFunction>:
sys[21]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

        10
  --------------
  s^2 + 7 s + 20
```

[4]:
```python
# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(L, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of OL System")
plt.show()

# Root locus = drawn for open loop system to see closed loop poles location at␣
 ↪s plane
ct.root_locus(L, grid=True, marker_size=10, marker_width=3)
plt.show()

# DC gain
dc = ct.dcgain(L)
print(f'DC gain for OL= {dc}')
# Zeros
z = ct.zeros(L)
print(f'Zeros of OL= {z}')
# Poles
p = ct.poles(L)
print(f'Poles of OL= {p}')

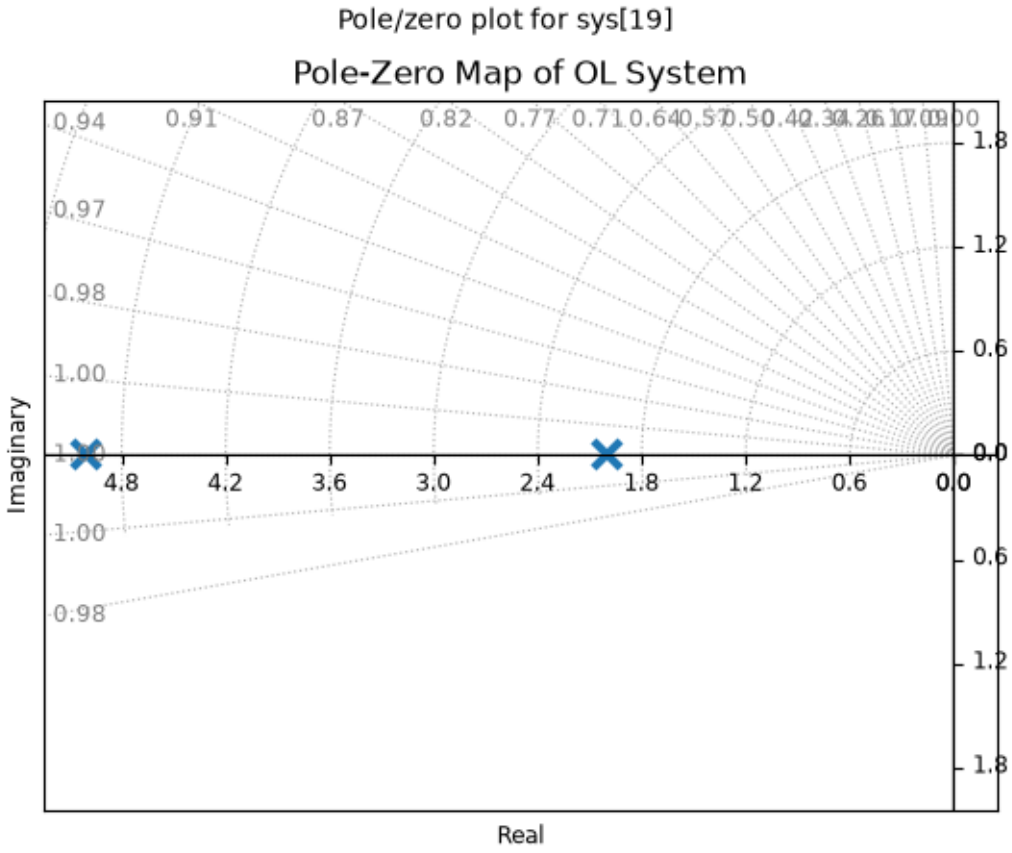
tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
```

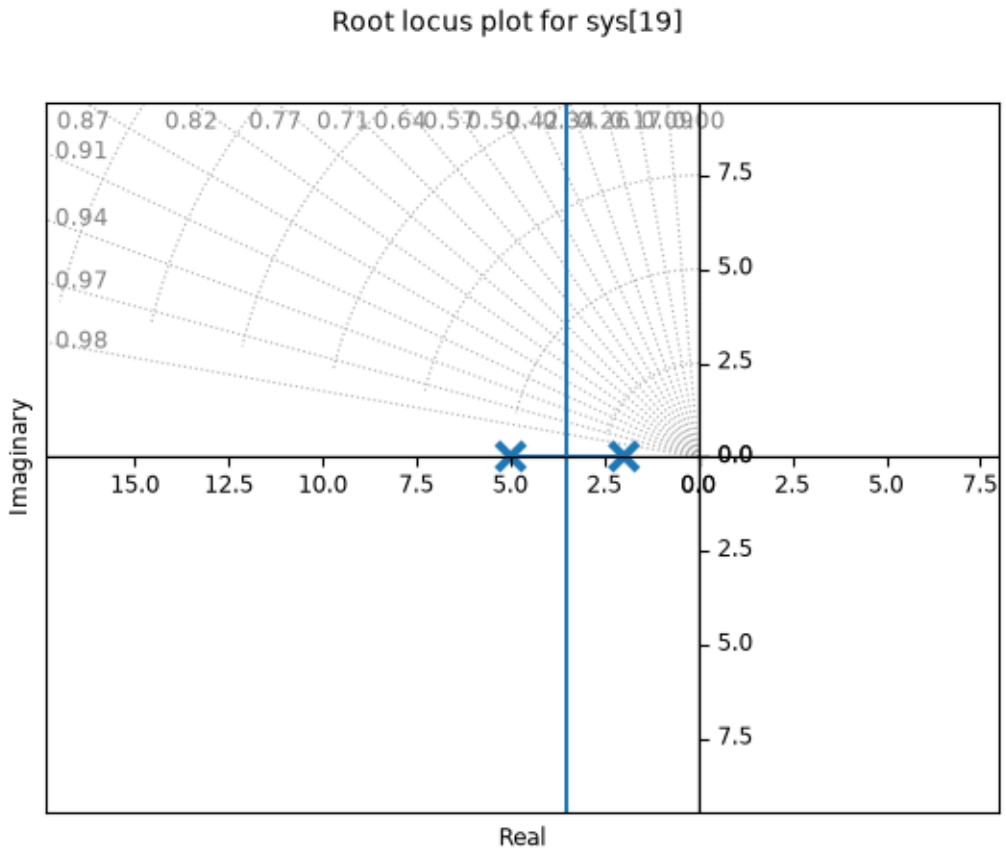
```
    print("No strictly stable pole -> no classical dominant pole.")

print("Dominant stable OL pole(s):", dominant_poles)

p1 = dominant_poles[0]
p_do = dominant_poles[0]
print(f'Dominant OL pole (p1) = {p1}')
print(f'Dominant OL pole (p1) in abs value = {np.abs(p1)}')
```




```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```




```
DC gain for OL= 1.0
Zeros of OL= []
Poles of OL= [-5.+0.j -2.+0.j]
Dominant stable OL pole(s): [-2.+0.j]
Dominant OL pole (p1) = (-2+0j)
Dominant OL pole (p1) in abs value = 2.0
```

[5]:
```python
# closed loop poles and zeros analysis

# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(CL, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of CL System")
plt.show()

# DC gain
dc = ct.dcgain(CL)
print(f'DC gain for CL= {dc}')
# Zeros
z = ct.zeros(CL)
print(f'Zeros of CL= {z}')
```

```
# Poles
p = ct.poles(CL)
print(f'Poles of CL= {p}')

# Print exact transient metrics derived from poles
wn, zeta, poles = ct.damp(CL)
print(f'Natural frequency of poles (Wn) = {wn}')
print(f'Damping ratio zeta of poles (Z) = {zeta}')
print(f'Poles = {poles}')

# To find the dominant pole(s) of a control system plant,
# you identify the poles (the roots of the denominator of the transfer␣
 ↪function) that lie closest to the imaginary (jw) axis
# or the origin in the s-plane,
# provided other poles are sufficiently far away
print(f'Poles of CL= {p}')

tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
    print("No strictly stable pole -> no classical dominant pole.")

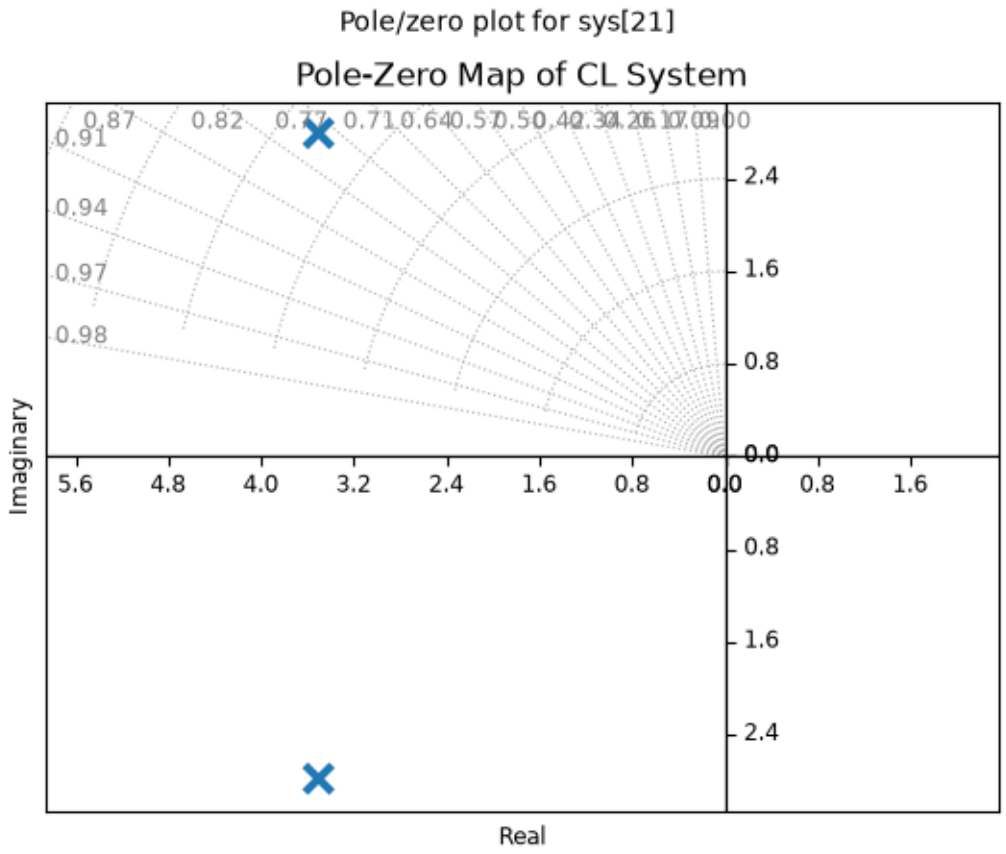
print("Dominant stable CL pole(s):", dominant_poles)

p1 = dominant_poles[0]
p_d = dominant_poles[0]
print(f'Dominant CL pole (p1) = {p1}')
print(f'Dominant CL pole (p1) in abs value = {np.abs(p1)}')
print('\n')

wn = abs(p1)                    # Natural frequency (rad/s)
zeta = -np.real(p1) / wn      # Damping ratio
sigma = -np.real(p1)          # Decay rate (1/s)
wd = abs(np.imag(p1))         # Damped frequency (rad/s)

print("Dominant Pole:", p1)
print("Natural frequency (wn):", wn, "rad/s")
print("Damping ratio (zeta, z):", zeta)
print("Decay rate (sigma):", sigma, "1/s")
```

```
print("Damped frequency (wd):", wd, "rad/s")
```




```
DC gain for CL= 0.5
Zeros of CL= []
Poles of CL= [-3.5+2.78388218j -3.5-2.78388218j]
    Eigenvalue (pole)       Damping     Frequency
      -3.5    +2.784j        0.7826         4.472
      -3.5    -2.784j        0.7826         4.472
Natural frequency of poles (Wn) = [4.47213595 4.47213595]
Damping ratio zeta of poles (Z) = [0.78262379 0.78262379]
Poles = [-3.5+2.78388218j -3.5-2.78388218j]
Poles of CL= [-3.5+2.78388218j -3.5-2.78388218j]
Dominant stable CL pole(s): [-3.5+2.78388218j -3.5-2.78388218j]
Dominant CL pole (p1) = (-3.5+2.783882181415011j)
Dominant CL pole (p1) in abs value = 4.47213595499958


Dominant Pole: (-3.5+2.783882181415011j)
Natural frequency (wn): 4.47213595499958 rad/s
Damping ratio (zeta, z): 0.7826237921249264
```

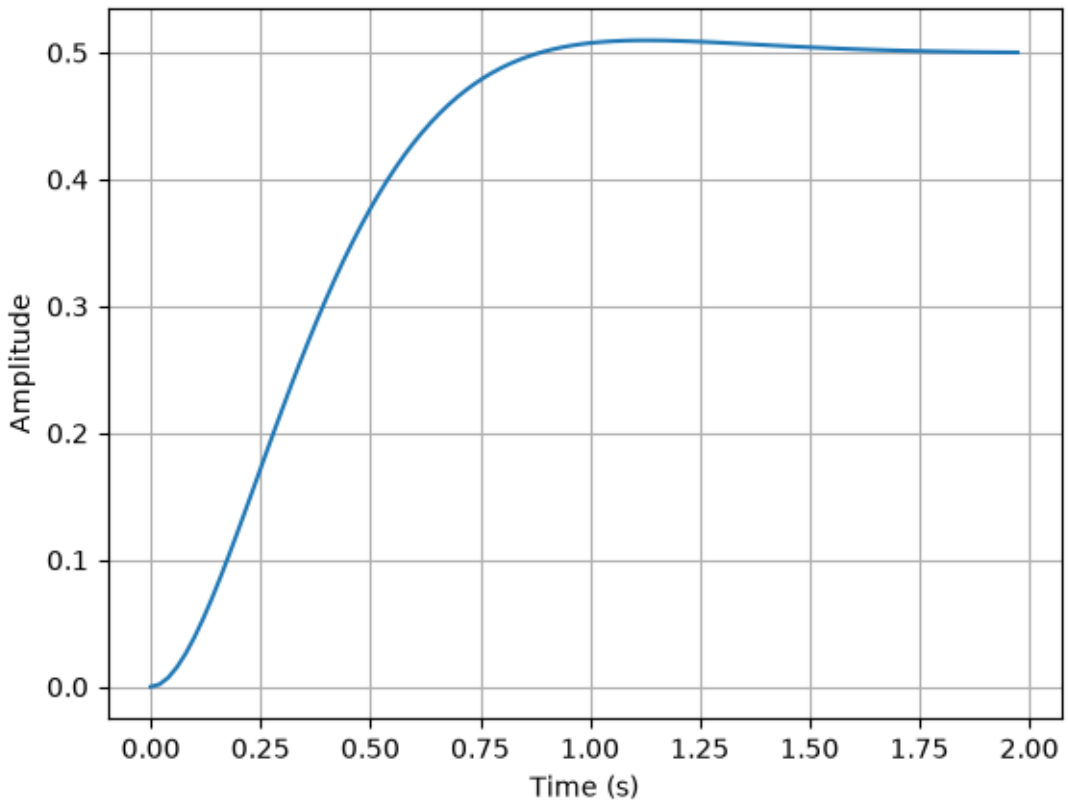
```
Decay rate (sigma): 3.5 1/s
Damped frequency (wd): 2.783882181415011 rad/s
```

[6]:
```python
plt.figure()
t, y = ct.step_response(CL)

plt.plot(t, y)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()

# transient characteristics
info = ct.step_info(CL)
final_value = info['SteadyStateValue']
ess = 1 - final_value


print(f"Rise Time (Tr): {info['RiseTime']:.2f} seconds")
print(f"Settling Time (Ts): {info['SettlingTime']:.2f} seconds")
print(f"Peak Time (Tp): {info['PeakTime']:.2f} seconds")
print(f"Overshoot (%): {info['Overshoot']:.2f}%")
# print(f"Undershoot (%): {info['Undershoot']:.2f}%")
print(f'Final Value = {info["SteadyStateValue"]}')
print(f'Steasy state error (ess) = {ess}')
```




```
Rise Time (Tr): 0.54 seconds
Settling Time (Ts): 0.82 seconds
Peak Time (Tp): 1.14 seconds
Overshoot (%): 1.92%
Final Value = 0.5
Steasy state error (ess) = 0.5
```

[7]:
```
# Type of the system = 1
# So, Kv is specified, Kv = lim:s->0 sG(s)Gc(s)

A1 = K*G2*G3
Kv_old = np.abs(A1(0))
print(f'Old velocity error constant (Kv_old) = {Kv_old}')

ess_old = 1/Kv_old
print(f'Old error (ess) = {ess_old}')
```

```
Old velocity error constant (Kv_old) = 2.0
Old error (ess) = 0.5
```

[8]:
```
# Required Specification
# ess = 0.01
```

[9]:
```
# Type of the system = 1
# So, Kv is specified, Kv = lim:s->0 sG(s)Gc(s)

ess_new = 0.01
print(f'Required error (ess) = {ess_new}')
Kv_new = 1 / ess_new
print(f'Required velocity error constant (Kv_new) = {Kv_new}')
```

```
Required error (ess) = 0.01
Required velocity error constant (Kv_new) = 100.0
```

[10]:
```
beta = Kv_new / Kv_old
print(f'Beta = {beta}')

# Nearest open loop pole
# -1/T = 10% of the nearest open or close loop pole
# P_do  np.real(p_do)
T = -1/(0.1*np.real(p_do))
print(f'T = {T}')

zc = 1/T
print(f'Zero of the lag compensator (zc) = {zc}')
pc = 1/(T*beta)
print(f'Pole of the lag compensator (pc) = {pc}')

# Lag Compensator transfer function without gain
Gc = ct.tf([1, zc], [1, pc])
print(f'Lead Compensator Transfer Function without gain (Gc) = {Gc}')

# |Gc(s)| = 1 and -5 < angle(Gc(s)) < 0
print(f'Open Loop Gain at required Pole Gc(Sd) = {Gc(p_do)}')
print(f'Open Loop Gain at required Pole in absolute form Gc(Sd) = {np.
 ↪abs(Gc(p_do))}')
print(f'Open Loop Phase at required Pole Gc(Sd) = {np.angle(Gc(p_do))}')
print(f'Open Loop Phase at required Pole in Degrees Gc(Sd) = {np.rad2deg(np.
 ↪angle(Gc(p_do)))}')
```

```
Beta = 50.0
T = 5.0
Zero of the lag compensator (zc) = 0.2
Pole of the lag compensator (pc) = 0.004
Lead Compensator Transfer Function without gain (Gc) = <TransferFunction>:
sys[30]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

   s + 0.2
  ---------
```

```
  s + 0.004
Open Loop Gain at required Pole Gc(Sd) = (0.9018036072144288-0j)
Open Loop Gain at required Pole in absolute form Gc(Sd) = 0.9018036072144288
Open Loop Phase at required Pole Gc(Sd) = -0.0
Open Loop Phase at required Pole in Degrees Gc(Sd) = -0.0
```

[11]:
```
# Lead compensator gain
Kc = 1/(np.abs(G(p_d)*Gc(p_d)))
print(f'Lead Compensator Gain (Kc) = {Kc}')
```

```
Lead Compensator Gain (Kc) = 1.0351135618070657
```

[12]:
```
Gc1 = Kc * Gc
print(f'Lead Compensator Transfer Function with gain (Gc1) = {Gc1}')

# |Gc(s)| = 1 and -5 < angle(Gc(s)) < 0
print(f'Open Loop Gain at required Pole G(Sd) = {Gc(p_d)}')
print(f'Open Loop Gain at required Pole in absolute form G(Sd) = {np.
 ↪abs(Gc(p_d))}')
print(f'Open Loop Phase at required Pole G(Sd) = {np.angle(Gc(p_d))}')
print(f'Open Loop Phase at required Pole in Degrees G(Sd) = {np.rad2deg(np.
 ↪angle(Gc(p_d)))}')
```

```
Lead Compensator Transfer Function with gain (Gc1) = <TransferFunction>: sys[32]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

  1.035 s + 0.207
  ---------------
     s + 0.004
Open Loop Gain at required Pole G(Sd) =
(0.9656911951202122-0.027320271902312825j)
Open Loop Gain at required Pole in absolute form G(Sd) = 0.9660775753476115
Open Loop Phase at required Pole G(Sd) = -0.028283354677068443
Open Loop Phase at required Pole in Degrees G(Sd) = -1.6205168534676193
```

[13]:
```
A2 = K*G2*G3*Gc1
# Type of system = 1 i.e. pole at origin = 1
# so the velocity error constant, Kv = lim:s->0 sG(s)Gc(s)
kv = np.abs(A2(0))
print(f'velocity error constant kv = {kv}')

ess = 1/kv
print(f'steady state error ess = {ess}')
```

```
velocity error constant kv = 103.51135618070657
steady state error ess = 0.009660775753476115
```

[14]:
```
A3 = G*Gc1
print(f'System Transfer Function with compensator A3(s) = {A3}')
A4 = A2 * H
print(f'open loop system transfer function with compensator A4(s) = {A4}')
A5 = ct.feedback(A3, 1)
print(f'Closed loop system transfer function with compensator A5(s) = {A5}')
```

```
System Transfer Function with compensator A3(s) = <TransferFunction>: sys[38]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

          10.35 s + 2.07
  --------------------------------
  s^3 + 7.004 s^2 + 10.03 s + 0.04
open loop system transfer function with compensator A4(s) = <TransferFunction>:
sys[40]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

     10.35 s + 2.07
  --------------------
  s^2 + 5.004 s + 0.02
Closed loop system transfer function with compensator A5(s) =
<TransferFunction>: sys[42]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

          10.35 s + 2.07
  --------------------------------
  s^3 + 7.004 s^2 + 20.38 s + 2.11
```

[15]:
```
# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(A4, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of OL System")
plt.show()

# Root locus = drawn for open loop system to see closed loop poles location at␣
 ↪s plane
ct.root_locus(A4, grid=True, marker_size=10, marker_width=3)
plt.show()

# DC gain
dc = ct.dcgain(A4)
print(f'DC gain for OL= {dc}')
# Zeros
z = ct.zeros(A4)
print(f'Zeros of OL= {z}')
```

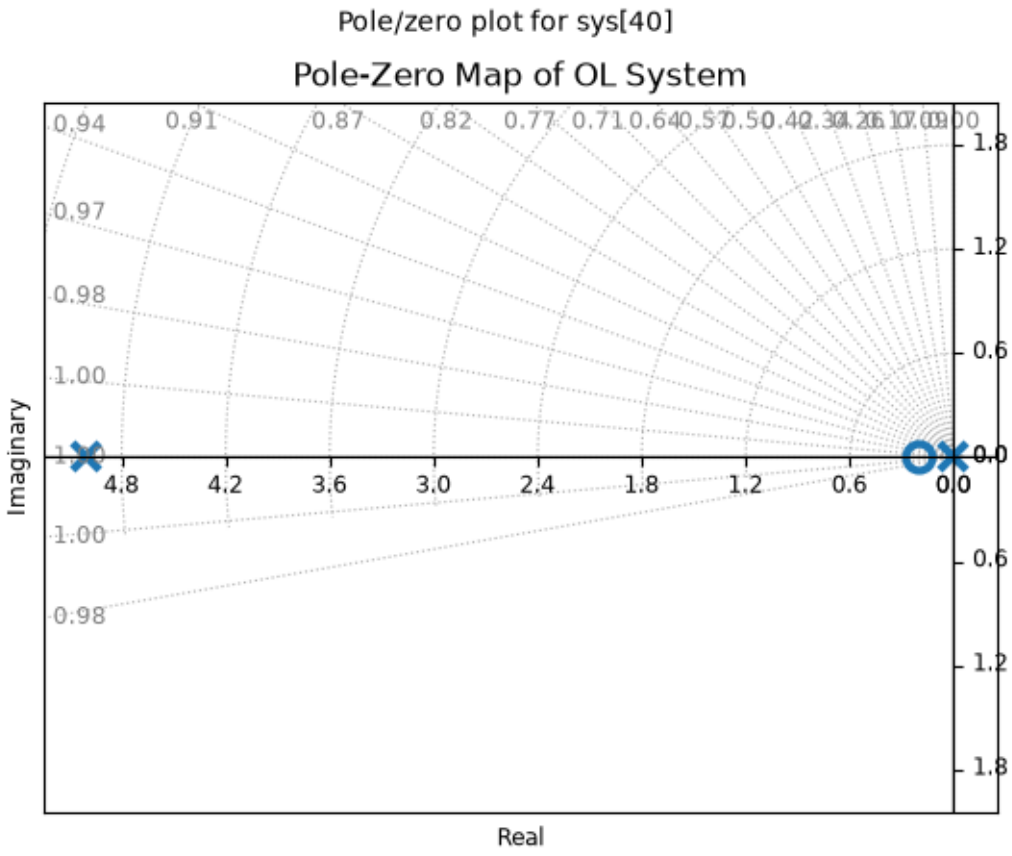
```python
# Poles
p = ct.poles(A4)
print(f'Poles of OL= {p}')

tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

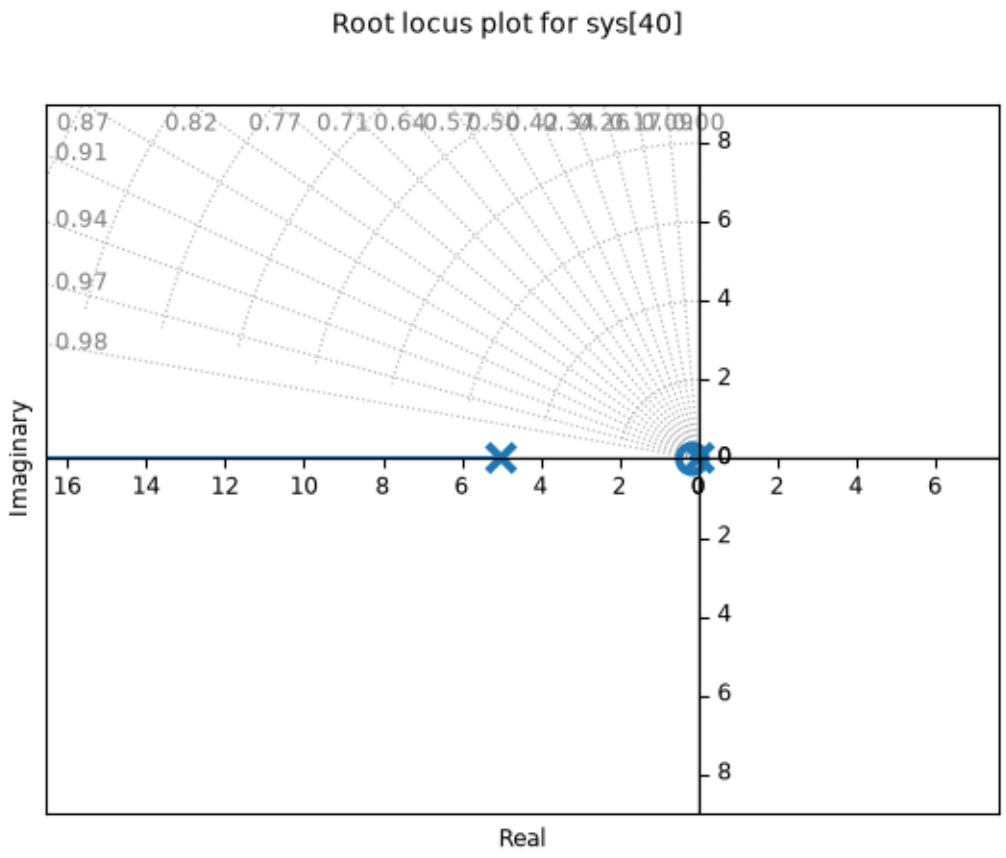
if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
    print("No strictly stable pole -> no classical dominant pole.")

print("Dominant stable OL pole(s):", dominant_poles)

p1 = dominant_poles[0]
print(f'Dominant OL pole (p1) = {p1}')
print(f'Dominant OL pole (p1) in abs value = {np.abs(p1)}')
```



```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```




```
DC gain for OL= 103.51135618070657
Zeros of OL= [-0.2+0.j]
Poles of OL= [-5.e+00+0.j -4.e-03+0.j]
Dominant stable OL pole(s): [-0.004+0.j]
Dominant OL pole (p1) = (-0.004+0j)
Dominant OL pole (p1) in abs value = 0.004
```

[16]:
```python
# closed loop poles and zeros analysis

# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(A5, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of CL System")
plt.show()

# DC gain
```

```
dc = ct.dcgain(A5)
print(f'DC gain for CL= {dc}')
# Zeros
z = ct.zeros(A5)
print(f'Zeros of CL= {z}')
# Poles
p = ct.poles(A5)
print(f'Poles of CL= {p}')

# Print exact transient metrics derived from poles
wn, zeta, poles = ct.damp(A5)
print(f'Natural frequency of poles (Wn) = {wn}')
print(f'Damping ratio zeta of poles (Z) = {zeta}')
print(f'Poles = {poles}')

# To find the dominant pole(s) of a control system plant,
# you identify the poles (the roots of the denominator of the transfer␣
 ↪function) that lie closest to the imaginary (jw) axis
# or the origin in the s-plane,
# provided other poles are sufficiently far away
print(f'Poles of CL= {p}')

tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

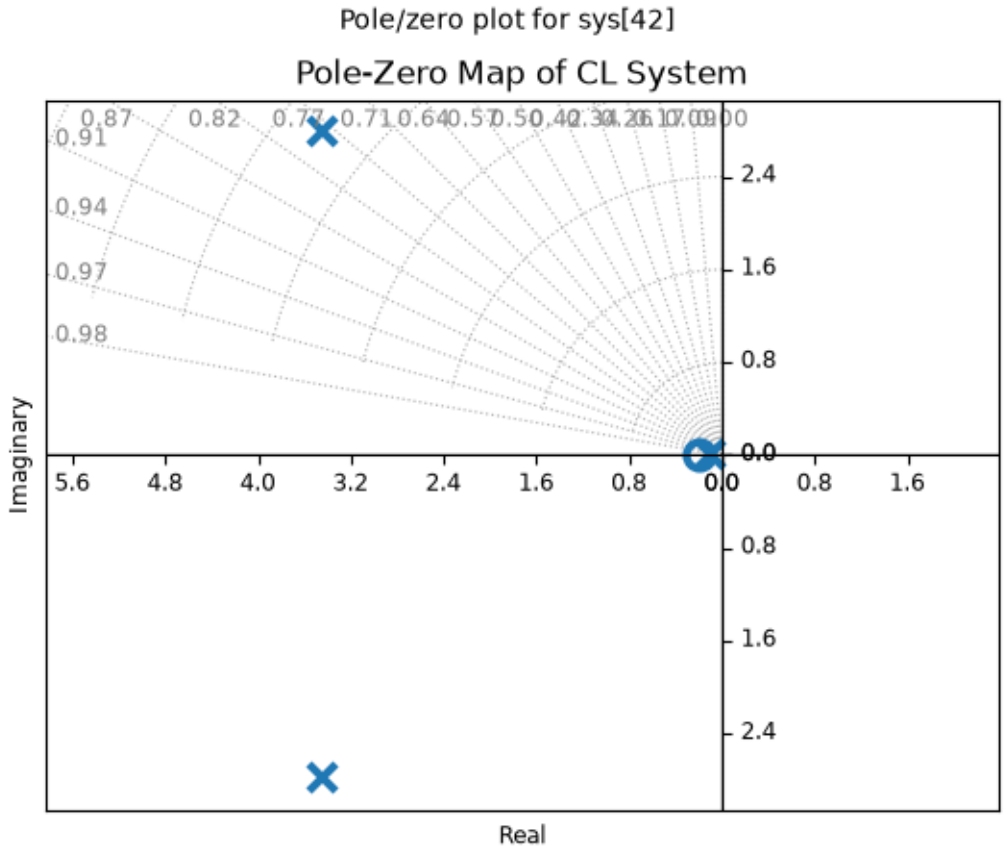
if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
    print("No strictly stable pole -> no classical dominant pole.")

print("Dominant stable CL pole(s):", dominant_poles)

p1 = dominant_poles[0]
print(f'Dominant CL pole (p1) = {p1}')
print(f'Dominant CL pole (p1) in abs value = {np.abs(p1)}')
print('\n')

wn = abs(p1)                    # Natural frequency (rad/s)
zeta = -np.real(p1) / wn      # Damping ratio
sigma = -np.real(p1)          # Decay rate (1/s)
wd = abs(np.imag(p1))         # Damped frequency (rad/s)
```

```
print("Dominant Pole:", p1)
print("Natural frequency (wn):", wn, "rad/s")
print("Damping ratio (zeta, z):", zeta)
print("Decay rate (sigma):", sigma, "1/s")
print("Damped frequency (wd):", wd, "rad/s")
```




```
DC gain for CL= 0.9810446944064044
Zeros of CL= [-0.2+0.j]
Poles of CL= [-3.44827201+2.78342969j -3.44827201-2.78342969j -0.10745598+0.j
]
    Eigenvalue (pole)       Damping     Frequency
    -3.448    +2.783j        0.7781         4.431
    -3.448    -2.783j        0.7781         4.431
              -0.1075             1        0.1075
Natural frequency of poles (Wn) = [4.43148516 4.43148516 0.10745598]
Damping ratio zeta of poles (Z) = [0.7781301 0.7781301 1.       ]
Poles = [-3.44827201+2.78342969j -3.44827201-2.78342969j -0.10745598+0.j
]
Poles of CL= [-3.44827201+2.78342969j -3.44827201-2.78342969j -0.10745598+0.j
]
```

```
Dominant stable CL pole(s): [-0.10745598+0.j]
Dominant CL pole (p1) = (-0.10745598335588433+0j)
Dominant CL pole (p1) in abs value = 0.10745598335588433

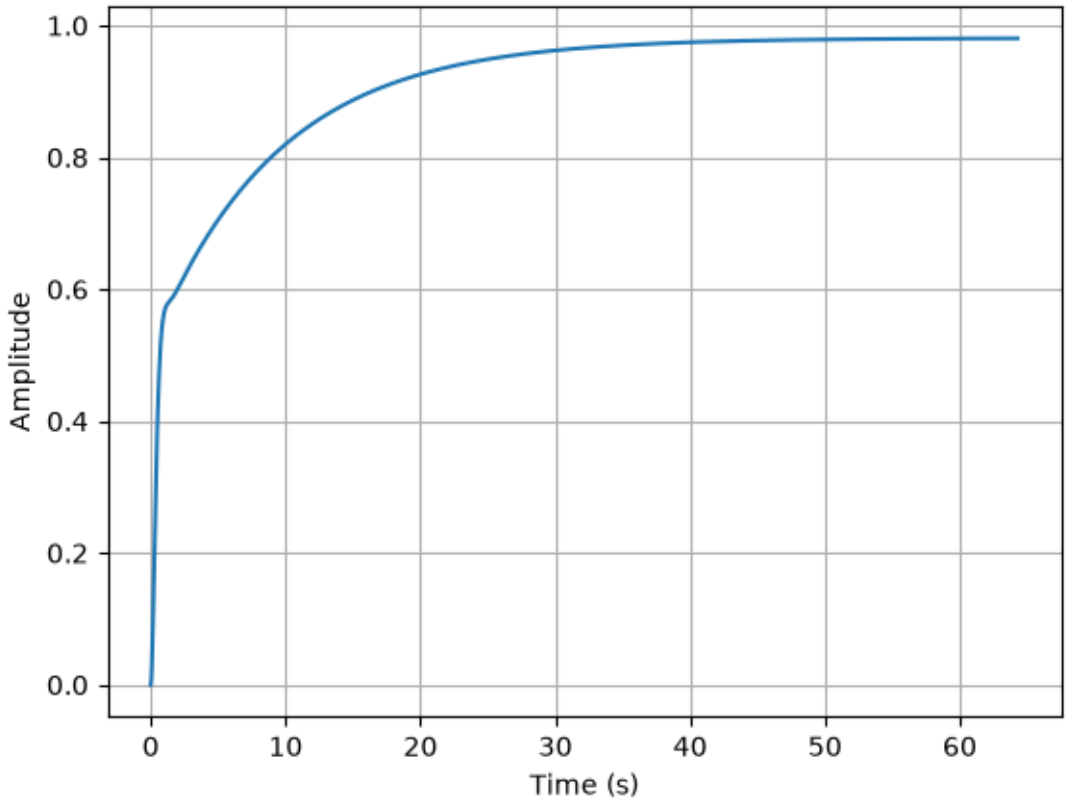

Dominant Pole: (-0.10745598335588433+0j)
Natural frequency (wn): 0.10745598335588433 rad/s
Damping ratio (zeta, z): 1.0
Decay rate (sigma): 0.10745598335588433 1/s
Damped frequency (wd): 0.0 rad/s
```

[17]:
```
plt.figure()
t, y = ct.step_response(A5)

plt.plot(t, y)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()

# transient characteristics
info = ct.step_info(A5)
final_value = info['SteadyStateValue']
ess = 1 - final_value

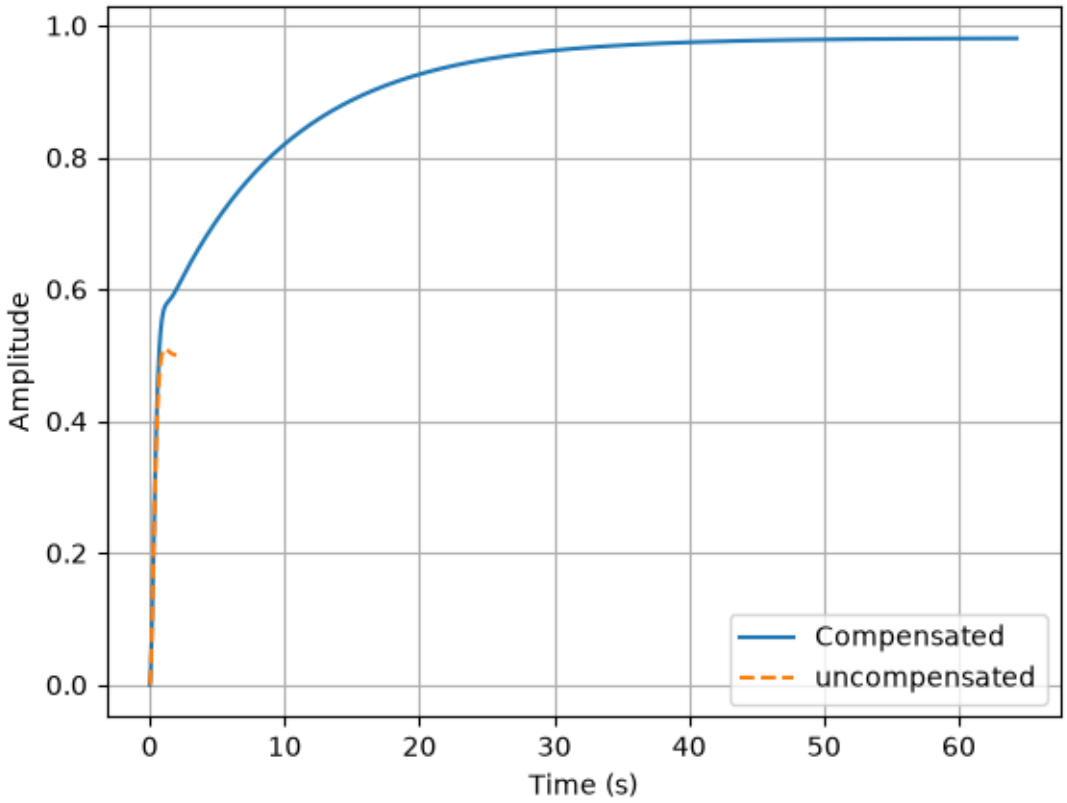

print(f"Rise Time (Tr): {info['RiseTime']:.2f} seconds")
print(f"Settling Time (Ts): {info['SettlingTime']:.2f} seconds")
print(f"Peak Time (Tp): {info['PeakTime']:.2f} seconds")
print(f"Overshoot (%): {info['Overshoot']:.2f}%")
# print(f"Undershoot (%): {info['Undershoot']:.2f}%")
print(f'Final Value = {info["SteadyStateValue"]}')
print(f'Steasy state error (ess) = {ess}')
```




```
Rise Time (Tr): 14.42 seconds
Settling Time (Ts): 29.60 seconds
Peak Time (Tp): 64.28 seconds
Overshoot (%): 0.00%
Final Value = 0.9810446944064044
Steasy state error (ess) = 0.018955305593595595
```

[18]:
```
plt.figure()
t1, y1 = ct.step_response(A5)
t2, y2 = ct.step_response(CL)

plt.plot(t1, y1, label='Compensated')
plt.plot(t2, y2, label='uncompensated', linestyle='--')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0.0
Amplitude
0
10
20
30
40
50
60
Time (s)
Compensated
uncompensated

[ ]: